Multi-Source Candidate Data Transformer — Design

Omkar Mishra · omkar794mishra@gmail.com · Eightfold Engineering Intern Assignment

1. Pipeline

detect → extract → combine → match/merge → infer → confidence → project → validate. *detect* classifies a raw input (file ext / URL host) into a SourceType without parsing content. *extract* is one module per source, each producing **Fragments** — partial canonical records where every field is wrapped as *Attributed<T>* = {value, source, method, confidence}, plus a list of *match hints* (normalized email, github handle, phone). *combine* stacks fragments in the same match group into one CanonicalCandidate (arrays of Attributed values per field, no winner chosen yet). *infer* fills gaps no source states directly (e.g. years_experience from earliest experience.start). *confidence* + *merge*'s resolve step pick one winner per field. *project* reads the resolved record into either the default schema or a runtime config. *validate* checks the projected output against a JSON Schema (default) or checks the config itself before projection runs (custom).

2. Canonical schema & normalized formats

Internal representation is richer than the output schema on purpose: every field keeps *all* candidate values (one per contributing source) until merge picks a winner, so provenance is never lost to early normalization. Chosen formats — **phone:** E.164 (bare 10-digit numbers assumed +91 for this dataset, already-international numbers passed through and re-validated, anything else → null, never guessed). **dates:** YYYY-MM (accepts "Jan 2021", "01/2021", "2021-01", bare year defaults to -01 at lower confidence; "Present"/"Current" → null end-date, meaning ongoing). **country:** ISO-3166 alpha-2 via a lookup table for common free-text forms; unrecognized text → null rather than a wrong guess. **skills:** lowercase + trim + an alias table ("ReactJS"/"react.js"/"React" → "react") so the same skill from different sources collapses to one canonical entry instead of three near-duplicates.

3. Merge & conflict-resolution policy

Matching: union-find over match keys (lowercased email, github username, phone) — not pairwise comparison — so matching is transitive: if source A and B share an email, and B and C share a GitHub handle, all three merge into one candidate even though A and C share nothing directly. Fragments with no match key at all become singleton candidates (conservative: never guess a merge with zero evidence).

Conflict resolution (one winner per scalar field): (1) highest *confidence* wins; (2) tie → higher *source priority* wins, ranked recruiter_csv > ats_json > linkedin > github > resume > recruiter_notes (an operator-entered field outranks an inferred one); (3) still tied → first-seen, deterministic. Array fields (emails, phones, skills) are de-duplicated *unions*, not single winners — a candidate legitimately has more than one of these.

Confidence: base score is set at extraction time per (source, method) pair — e.g. a recruiter CSV's direct_field starts higher than a GitHub bio's heuristic_inference. At merge time, the winning value's score is then adjusted: *+bonus per additional source that agrees* (capped), *−penalty if any source disagreed* even though it lost — a contested field is never fully trusted just because we picked an answer. Final score is capped at 0.98: the system never claims certainty, directly addressing the brief's "wrong-but-confident is worse than honestly-empty."

4. Runtime custom-output config

Clean separation: the internal CanonicalCandidate/ResolvedCandidate never changes shape based on config. A *projection* layer reads a flattened view of the resolved record through a small path language (dot-nesting, [n] fixed index, [] map-over-array, e.g. "skills[].name") driven by the config's field list. Each field can set a normalize kind (E164 / ISO-3166 / canonical) applied at projection time, so the same internal value can be re-shaped differently per config without re-running extraction. on_missing is handled per the three documented modes (null / omit / error) at the point a field resolves to nothing. The config itself is schema-checked before any candidate is projected, so a typo'd config fails fast with one clear error instead of N silent per-candidate failures.

5. Edge cases handled (and what's deliberately left out)

- **Malformed row** (wrong cell count in CSV) → skipped with a warning; run continues. Live in sample-data.
- **Unreachable/rate-limited API source** (e.g. GitHub's 60 req/hr unauthenticated cap) → caught, warned, skipped, never throws — proven live against the real GitHub API in the sample run.
- **Same person, different shared key per source** → handled by transitive union-find matching (see §3).
- **Conflicting field values across sources** → resolved by confidence + source priority, with the loser still discounting the winner's confidence (see §3).
- **Unnormalizable garbage value** (e.g. phone "not-a-phone") → normalizer returns null, never a fabricated or passed-through bad value.

Deliberately descoped under the time box: ATS JSON, resume PDF/DOCX, and recruiter notes extractors (same Fragment contract as the two implemented sources — additive, not architecture-changing) and LinkedIn (no public/ToS-compliant API for an assignment-scale project). Also descoped: fuzzy name+company matching as a fallback when no shared identifier exists — current behavior is conservative (no merge without an explicit shared key), which I chose over a recall-favoring fuzzy match that risks merging two different people, given the brief's explicit warning that a wrong-but-confident value is worse than an empty one.